Week 2 Design & Testing Report — MLFQ Scheduler

Project: Multi-Level Feedback Queue (MLFQ) Scheduler
Team Members: Marium Ali Lakhani (29183), Fatima Faisal (28989)
Date: 26th Nov 2025

1. Introduction

This week focuses on extending the MLFQ scheduler implemented in Week 1 to fully enforce time slices, handle demotion, and implement global priority boosting.
The goal is to ensure CPU-bound processes are demoted across queues, I/O-bound processes remain responsive, and starvation is prevented through periodic boosts.

2. Scheduler Enhancements

2.1 Time-Slice Enforcement

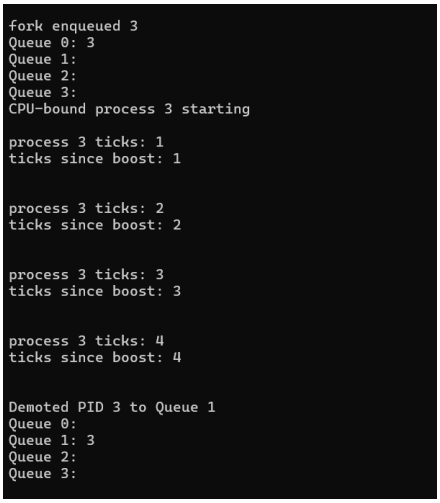
- Each queue has a fixed quantum:
  - **Q0:** 4 ticks
  - **Q1:** 8 ticks
  - **Q2:** 16 ticks
  - **Q3:** ∞ (never demotes)
- trap.c increments p->time_in_queue on every timer interrupt.
- When a process exceeds its time slice, it yields back to the scheduler.

2.2 Demotion

- CPU-bound processes that exhaust their time slices are demoted one queue level at a time.
- Expected Behaviour:

Queue	Time slice	Result on exhaustion
Q0	4 ticks	Demote to Q1
Q1	8 ticks	Demote to Q2
Q2	16 ticks	Demote to Q3
Q3	∞	No demotion

- Observed behavior:



- Scheduler logs demotion events with printf() and updates queue states.

2.3 Global Priority Boost

- A counter ticks_since_boost tracks elapsed ticks.
- When ticks_since_boost >= BOOST_TIME (40 ticks in this implementation), all processes in lower queues (Q1–Q3) are boosted to Q0.
- Time counters time_in_queue are reset.
- This prevents starvation of long-running CPU-bound processes.

3. Queue Management

- Implemented as circular queues (struct proc_queue) for each priority level.
- Functions:
    - enqueue() — add process to tail
    - dequeue() — remove process from head
    - is_empty() — check if queue is empty
    - print_queue() — prints queue contents for debugging
- Locks (spinlock) used per queue to ensure thread safety.
- All queues are initialized in procinit() with head = tail = 0 and procs[] = NULL.

4. Testing Environment

- **Tools:** QEMU (xv6 simulator), terminal output logs.
- **Programs:**
    - io — I/O-bound process (yields frequently)
    - burn — CPU-bound process (busy loop)
    - quick — short-running process (stays in high-priority queue)
- **Observation Method:**
    - Scheduler debug prints (process ticks, Demoted PID X, Queue state)
    - Periodic print_queue() calls to monitor queue contents.

5. Test Cases & Observations

5.1 I/O-Bound Process

- Run: i.co
- **Result:**
    - Process stays in Q0
    - No demotion occurs
    - Responsive behavior confirmed

5.2 CPU-Bound Process

- Run: $ burn (infinite loop)
- **Result:**
    - Process gradually moves from Q0 → Q1 → Q2 → Q3
    - Time-slice enforcement works correctly
    - Queue printing reflects demotions